DxPS

Real-time Provisioning System

Components and Data Guide

PostgreSQL structure and examples - Kafka topics and messages - services

18	41	36	303 ms
PostgreSQL tables	Kafka topics	consumer groups	example order

Item	Value
Document	DXPS-COMPONENTS-GUIDE
Version	1.1
Date	17 October 2026
Applies to	dxps source of the same date, including the DxPS console; PostgreSQL 18, Apache Kafka 4.3.1, Go 1.26.6
Examples	Real records read with dxpsctl trace from the local run of 06 October 2026

Table of Contents

1. Purpose and component map 3
2. One order through every component 4
3. PostgreSQL 18 - system of record 5
3.1 What PostgreSQL is used for 5
3.2 Database, schema and roles 5
3.3 The tenant model 5
3.4 Table catalogue 6
3.5 Partitioning 7
3.6 Real rows 7
3.7 A real rollback (saga compensation) 8
3.8 Concurrency patterns 8
3.9 Useful queries 9
4. Apache Kafka 4 - asynchronous transport 10
4.1 What Kafka is used for 10
4.2 Topic catalogue 10
4.3 Keys and headers 11
4.4 Messages (real) 11
4.5 Producers 12
4.6 Consumers 12
4.7 Outbox relay: database and Kafka in step 12
4.8 Volumes after the 2,000-order load run 12
5. The other components 13
5.1 Gateway (northbound API) 13
5.2 Orchestrator 13
5.3 Catalog 13
5.4 Adapter and network elements 13
5.5 Network simulators 14
5.6 Event hub 14
5.7 Mosaic dashboard 14
5.8 DxPS console 14
5.9 PKI, secrets and dxpsctl 17
6. What happens when something fails 18

1. Purpose and component map

This guide explains what each DxPS component is responsible for, how it is used and what data it holds. It covers PostgreSQL (tables, keys, tenancy, partitions), Apache Kafka (topics, keys, headers, messages, consumer groups) and the DxPS services around them. Every example row and message is real: it was read from the running system with dxpsctl trace: order 1f2cd4a6-6333-5fbc-8fe1-0979d5966478 (tenant host-mno, a subscriber plus VoNR, submitted from the Mosaic load generator through the live services), plus an order that was rolled back in the end-to-end tests.

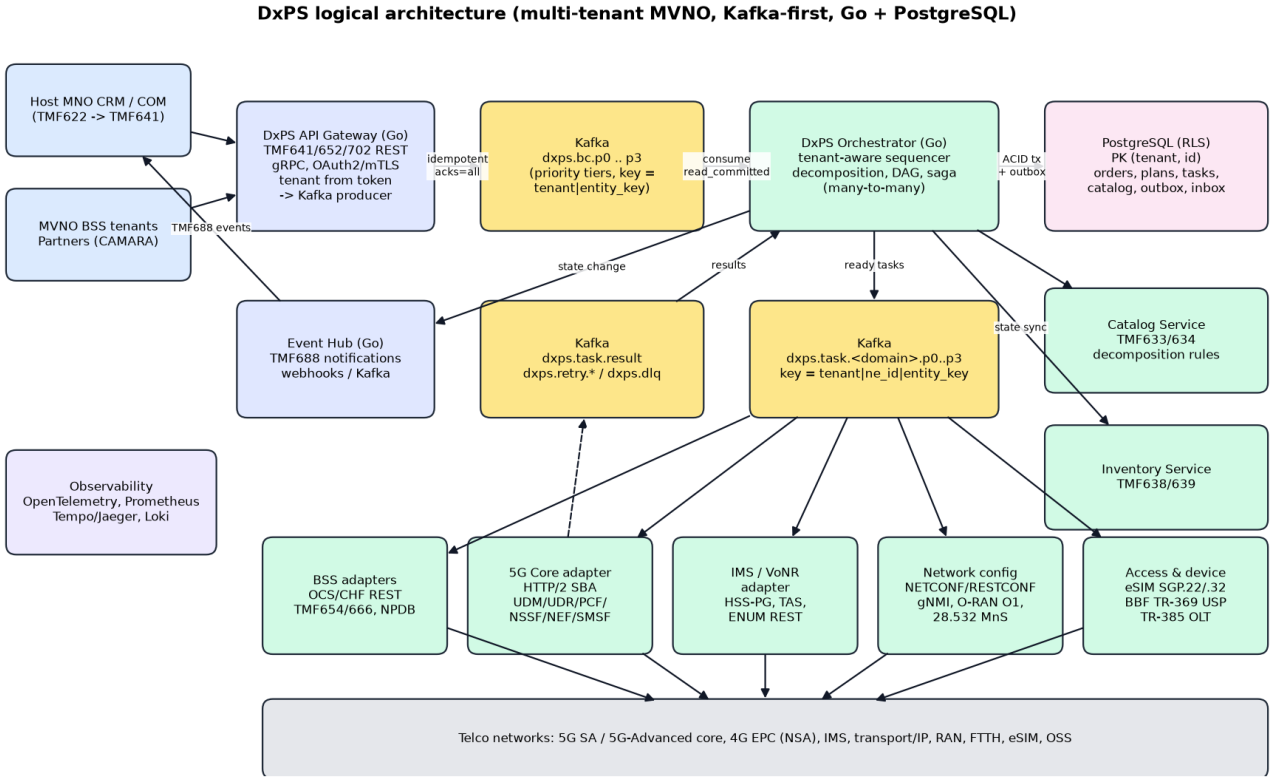


Figure: DxPS logical architecture

Component	Technology	Responsibility	Code
Gateway	Go, HTTP/2, TLS 1.3	TMF641 order API and TMF688 hub; authenticates the tenant; stores the order and its business commands in one transaction	internal/gateway
Orchestrator	Go	Plans each business command into a DAG of NE tasks; runs the DAG; retries; saga compensation; order state	internal/orchestrator, planner, saga, scheduler, keylane
Adapter	Go, HTTP/2 mTLS	Renders NE payloads and calls the network elements for 7 domains; NE budgets and circuit breakers	internal/adapter, registry, breaker
Event hub	Go	Delivers TMF688 state-change events to tenant webhooks, HMAC-signed	internal/eventhub, secretbox
PostgreSQL 18	pgx v5	System of record: tenants, NE inventory, catalog, orders, plans, tasks, attempts, audit, outbox	internal/store, migrations/001_schema.sql
Apache Kafka 4 (KRaft)	franz-go	Asynchronous transport between services, priority lanes, retry ladder, events, dead letters	internal/bus, internal/contract
Catalog	YAML + CEL	Service recipes: tasks, dependencies, conditions, compensations, payload templates; per-tenant overrides	internal/catalog/data
Network simulators	Go, HTTP/2 mTLS	Stand-ins for NRF, UDR, IMS, RESTCONF/USP, SM-DP+, OCS, NPDB, NEF; fault injection	internal/netsim
Mosaic dashboard	Go + HTML/JS	Live operations view and load generator	internal/dashboard
DxPS console	Go API + Next.js 16 UI	Signed-in operations and configuration console: orders, DLQ, simulators, catalog drafts, Template Lab, tenants, NE inventory, reports, users	cmd/console, internal/console, console/
dxpsctl	Go CLI	Secrets, PKI, migrations, seed, topics, tokens, DLQ re-drive, API calls, trace, test report	cmd/dxpsctl

2. One order through every component

The timeline below is the real life of order 1f2cd4a6-6333-5fbc-8fe1-0979d5966478 (external id DEMO-HOST-MNO-8030236): CreateSubscriber5G plus AddVoNR, where the VoNR item dependsOn the subscriber item. The plan is LTE-20GB, so the catalog condition params.plan.startsWith('5G') removed the policy task and the planner created 4 tasks instead of 5. The order completed in 303 ms.

Time (20:44:..)	Component	What happened	Where it is recorded
10.346	Gateway	Order accepted (HTTP 201). One transaction writes the service_order row and two outbox rows (one BusinessCommand per order item)	service_order, outbox
10.352	Gateway -> Kafka	Both BusinessCommands published to dxps.bc.p1 (fast path, 6 ms after commit), key host-mno|supi:imsi-416771008030236	outbox.published_at
10.353	Orchestrator	Inbox dedupe; the entity sequencer admits item 'sub'; the planner creates 4 ne_task rows and their dependency edges; AddVoNR waits for 'sub'. The order moves to inProgress	orchestrator_inbox, business_command, entity_sequencer, ne_task, task_dependency, bc_task_link, service_order
10.385	Orchestrator -> Kafka	First NeTask (udr.authSubscription.put) to dxps.task.sba.p1; TMF688 inProgress event to dxps.event.tmf688 and dxps.state.order	outbox
10.395	Adapter -> udr01	PUT authentication-subscription, HTTP 201 in 3.8 ms; the TaskResult and the consumed offset are committed in one Kafka transaction	task_attempt
10.442 - 10.463	Orchestrator / Adapter	am-data and smf-selection run in parallel once auth succeeded	ne_task, task_attempt
10.499	Adapter -> ocs01	OCS account created after both am and smf (pivot task)	ne_task, task_attempt
10.507 - 10.649	Orchestrator / Adapter	Item 'sub' completed, so AddVoNR starts on imspg01: IMPI -> IMPU -> MMTEL, strictly in order	business_command, ne_task, task_attempt
10.649	Orchestrator	Both business commands completed: order completed; the subscriber's sequencer is released (next_seq 3)	service_order, entity_sequencer, outbox
10.673	Orchestrator -> Kafka -> Event hub	completed event published; the event hub posts it, signed, to the host-mno webhook	dxps.event.tmf688, dxps.state.order

3. PostgreSQL 18 - system of record

3.1 What PostgreSQL is used for

- Everything that must survive a restart lives in PostgreSQL: tenants, NE inventory and entitlements, the catalog, orders, business commands, the task DAG, every NE attempt, audit and the outbox.
- Kafka is never the source of truth. If a Kafka message is lost or duplicated, the database state decides what happens (inbox dedupe, task attempt numbers, idempotency keys).
- Every state change and the messages it causes are committed in one transaction (transactional outbox). A crash can never leave a task marked dispatched without its Kafka message, or the other way round.

3.2 Database, schema and roles

One database dxps with one schema dxps, on localhost:5433. Connections require TLS 1.3 and SCRAM-SHA-256; pg_hba.conf contains only hostssl loopback entries. Three login roles separate duties:

Role	Used by	Rights	Row-level security
dxps_owner	dxpsctl migrate	Owns the schema; runs migrations	owner (not used at runtime)
dxps_app	Gateway, orchestrator, event hub, dxpsctl trace	SELECT/INSERT/UPDATE; no UPDATE on append-only tables; DELETE only on hub_subscription	Enforced: sees only the tenant of the current transaction
dxps_ops	Outbox relay, registry loader, seeding, dashboard statistics	Read all; write inventory/catalog; mark outbox rows published	BYPASSRLS (cross-tenant by design)

3.3 The tenant model

Every table starts with tenant (domain dxps_tenant = varchar(40) with a format check) and a UUIDv7 id; the primary key is always (tenant, id) and every foreign key includes the tenant, so a row can never point at another tenant's row.

```
CREATE DOMAIN dxps_tenant AS varchar(40) CHECK (VALUE ~ '^[A-Za-z0-9_.-]{1,40}$');

CREATE TABLE ne_task (
  tenant dxps_tenant NOT NULL, id uuid NOT NULL DEFAULT uuidv7(),
  order_id uuid NOT NULL, ne_tenant dxps_tenant NOT NULL, ne_id uuid NOT NULL,
  ...
  PRIMARY KEY (tenant, id),
  FOREIGN KEY (tenant, order_id)  REFERENCES service_order (tenant, id),
  FOREIGN KEY (ne_tenant, ne_id)  REFERENCES network_element (tenant, id)
) PARTITION BY LIST (tenant);

-- applied to all 16 tenant tables
CREATE POLICY tenant_isolation ON ne_task
  USING (tenant = current_tenant()) WITH CHECK (tenant = current_tenant());
```

The application never filters by tenant by hand. store.InTenant opens a transaction and sets the transaction-local setting app.tenant; current_tenant() reads it and the policy does the rest. If it is not set, current_tenant() is NULL and no rows are visible.

```
func (s *Store) InTenant(ctx context.Context, tenantID string, fn func(pgx.Tx) error) error {
    return pgx.BeginTxFunc(ctx, s.App, pgx.TxOptions{IsoLevel: pgx.ReadCommitted}, func(tx pgx.Tx) error {
        if _, err := tx.Exec(ctx, "SELECT set_config('app.tenant', $1, true)", tenantID); err != nil {
            return err
        }
        return fn(tx)
    })
}
```

Note: Real effect: in a trace of an mvno-beta order (d9e8f3cb-2cf1-5f23-8429-8781c9eeb0d1) the NE code column is empty. The NE udr01 belongs to host-mno, so mvno-beta's transaction cannot read the network_element row, even though its own tasks reference it through (ne_tenant, ne_id). The catalog is the one deliberate exception: command_spec rows of tenant GLOBAL are readable by everybody.

3.4 Table catalogue

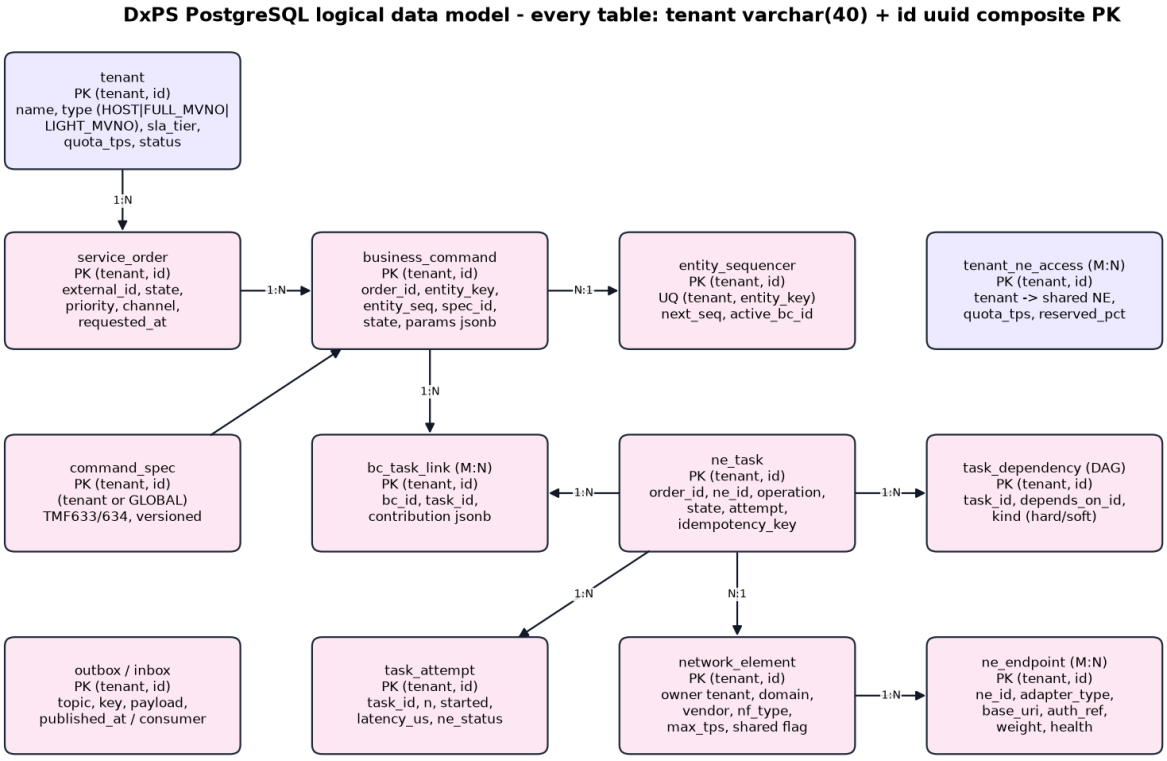


Figure: Entity relationships

Table	Purpose	Notable columns	Partitioning
tenant	Tenants: host MNO, MVNOs, enterprises	tenant_type, host_tenant, sla_weight, quota_tps, status	-
network_element	NE inventory, owned by a tenant	code, domain, nf_type, shared, max_tps, max_concurrency, reserved_pct	-
ne_endpoint	How to reach an NE (https only)	adapter_type, base_uri, auth_ref (vault reference, not a secret)	-
tenant_ne_access	Which tenant may use which NE, for which operations	quota_tps, allowed_ops[], subscriber_group	-
command_spec	Catalog recipes as loaded from YAML (GLOBAL or tenant override)	spec_code, version, status, body jsonb	-
service_order	TMF641 order	state, priority, tx_mode, idempotency_key, request_hash, items jsonb, version	LIST(tenant)
business_command	One per order item: what to do to which entity	entity_key, entity_seq, spec_code/version, params, depends_on_items	LIST(tenant)
bc_entity	Every entity key a command touches (supi, msisdn)	entity_key, bc_id	LIST(tenant)
entity_sequencer	Serialises commands per subscriber / resource	entity_key, next_seq, active_bc_id	LIST(tenant)
ne_task	One NE operation in the DAG	operation, state, attempt, in_degree, is_compensation, idempotency_key, coalesce_key, request, response	LIST(tenant)
task_dependency	DAG edges	task_id, depends_on_id, kind	LIST(tenant)
bc_task_link	Which command produced which task (tasks can be shared)	bc_id, task_id, contribution	LIST(tenant)
task_attempt	Append-only log of every NE call	n, started_at, latency_us, ne_status, ne_code, outcome	RANGE(id) daily
audit_event	Append-only audit trail	actor, acting_for_tenant, action, object	RANGE(id) daily
outbox	Kafka messages waiting to be published	shard, topic, key, payload, headers, published_at	-
orchestrator_inbox	Message ids already	id = Kafka message_id	RANGE(id) daily

	processed (dedupe)		
hub_subscription	TMF688 webhook registrations	callback (https), query, secret_ref (encrypted)	-

3.5 Partitioning

- LIST by tenant for the busy order and orchestration tables. Large tenants get a dedicated partition (service_order_host_mno, service_order_mvno_alpha); everybody else shares a DEFAULT partition that is itself HASH-partitioned four ways (service_order_shared_00 .. _03). The traces show host-mno rows in service_order_host_mno and mvno-beta rows in service_order_shared_01.
- RANGE by id for the append-only tables. Ids are UUIDv7, so they sort by time; uuid_floor(ts) gives the lowest UUIDv7 of a timestamp and is used as the partition bound. One partition per day (task_attempt_20261006) plus a default safety net; old days can be detached and archived without touching live data.
- Fill factors are tuned per table: 70 for the hot entity_sequencer, 80 for ne_task, 90 for orders, 100 for insert-only links.

3.6 Real rows

service_order (trimmed; items holds the TMF641 order items as received):


```
{ "partition": "service_order_host_mno", "tenant": "host-mno",
  "id": "1f2cd4a6-6333-5fbc-8fe1-0979d5966478", "external_id": "DEMO-HOST-MNO-8030236",
  "channel": "DASHBOARD", "state": "completed", "priority": 1, "tx_mode": "ATOMIC", "version": 2,
  "idempotency_key": "demo-01a11476-2868-7647-9fe0-2121b3bf35f9",
  "requested_at": "2026-10-06T20:44:10.346741-07:00", "completed_at": "2026-10-06T20:44:10.649652-07:00",
  "items": [ {"id": "sub",   "service": {"serviceSpecification": {"id": "CreateSubscriber5G"}, ...}},
             {"id": "volte", "service": {"serviceSpecification": {"id": "AddVoNR"}, ...},
              "serviceOrderItemRelationship": [{"relationshipType": "dependsOn", "orderItem": {"itemId": "sub"}}]} ] }
```


business_command - one row per order item; the second one waits for the first:

order_item_id	spec (tenant / version)	entity_key	params	depends_on_items
sub	CreateSubscriber5G (GLOBAL / 3.2.0)	supi:imsi-416771008030236	supi, msisdn 962718030236, plan LTE-20GB	-
volte	AddVoNR (GLOBAL / 1.1.0)	supi:imsi-416771008030236	supi, msisdn	sub

ne_task - one row per NE operation. The request and response columns keep the exact payloads exchanged with the NE:


```
{ "partition": "ne_task_host_mno", "tenant": "host-mno", "id": "01a11476-287d-7ed4-89e1-aaa76229497b",
  "ne_tenant": "host-mno", "ne_id": "01a113c4-d818-7b24-bb32-a35af5896bea", "ne_code": "udr01",
  "domain": "sba", "operation": "udr.authSubscription.put",
  "state": "SUCCEEDED", "attempt": 0, "in_degree": 0, "priority": 1, "is_compensation": false,
  "idempotency_key": "host-mno:01a11476-287d-7ed4-89e1-aaa76229497b:0",
  "request": { "supi": "imsi-416771008030236", "authenticationMethod": "5G_AKA", "algorithmId": "milenage",
               "encPermanentKey": "hsm://host-mno/k/imsi-416771008030236",
               "encOpcKey": "hsm://host-mno/opc/imsi-416771008030236",
               "sequenceNumber": {"sqn": "000000000020", "sqnScheme": "NON_TIME_BASED"}, ... },
  "ne_status": 201,
  "created_at": "20:44:10.353", "dispatched_at": "20:44:10.353", "completed_at": "20:44:10.399" }
```


Note: Key material is never stored: the payload carries HSM references (hsm://tenant/k/...), not keys.

task_dependency (the DAG) and task_attempt (one row per NE call, partition task_attempt_20261006) for the same order:

Task	NE	Depends on	NE status / latency	Started
udr.authSubscription.put	udr01	-	201 / 3.8 ms	10.395
udr.amData.put	udr01	udr.authSubscription.put	201 / 4.8 ms	10.442
udr.smfSelection.put	udr01	udr.authSubscription.put	201 / 3.9 ms	10.459
bss.account.create	ocs01	udr.amData.put +	201 / 7.7 ms	10.499

		udr.smfSelection.put		
ims.impi.put	imspg01	(item 'sub' completed)	200 / 5.5 ms	10.552
ims.impu.put	imspg01	ims.impi.put	200 / 6.2 ms	10.595
ims.mmtel.put	imspg01	ims.impu.put	200 / 6.2 ms	10.643

entity_sequencer - the subscriber's lane. Both commands for imsi-416771008030236 ran through it in order; active_bc_id is empty again because nothing is running:

```
{ "partition": "entity_sequencer_host_mno", "tenant": "host-mno",
  "entity_key": "supi:imsi-416771008030236", "next_seq": 3, "active_bc_id": null,
  "updated_at": "2026-10-06T20:44:10.649652-07:00" }
```

outbox - the 13 Kafka records this order produced, in commit order. Each was published 6 - 35 ms after its transaction committed:

Topic	Key	schema_id	Count
dxps.bc.p1	host-mno|supi:imsi-416771008030236	dxps.v1.BusinessCommand	2
dxps.task.sba.p1	host-mno|<udr01 id>|supi:imsi-416771008030236	dxps.v1.NeTask	3
dxps.task.bss.p1	host-mno|<ocs01 id>|supi:imsi-416771008030236	dxps.v1.NeTask	1
dxps.task.ims.p1	host-mno|<imspg01 id>|supi:imsi-416771008030236	dxps.v1.NeTask	3
dxps.event.tmf688	host-mno|1f2cd4a6-...	tmf688.ServiceOrderStateChangeEvent	2 (inProgress, completed)
dxps.state.order	host-mno|1f2cd4a6-...	tmf688.ServiceOrderStateChangeEvent	2

3.7 A real rollback (saga compensation)

Order 44a48169-3aae-50a2-9e8a-455f61bc205d (mvno-beta, CreateSubscriber5G, ATOMIC), from the end-to-end test TestCompensationE2E. The UDR rejected the AM data. The OCS step had not started, so it was skipped; the three steps that had succeeded were undone in reverse order, and the order failed with DXPS-3000. A dead-letter audit record was written to dxps.dlq.

Task	State	NE status / code	Done at
udr.authSubscription.put	COMPENSATED	201	17.377
udr.amData.put	FAILED	400 DXPS-3400:MANDATORY_IE_INCORRECT	17.441
udr.smfSelection.put	COMPENSATED	201	17.464
udr.policyData.put	COMPENSATED	201	17.519
bss.account.create	SKIPPED	-	17.441
udr.policyData.delete (compensation)	SUCCEEDED	204	17.552
udr.smfSelection.delete (compensation)	SUCCEEDED	204	17.575
udr.authSubscription.delete (compensation)	SUCCEEDED	204	17.653

3.8 Concurrency patterns

- Optimistic versioning on service_order (version column) and row locks (SELECT ... FOR UPDATE) when the order state is derived.
- Outbox relay: SELECT ... ORDER BY shard, id LIMIT 500 FOR UPDATE SKIP LOCKED, so several relays never publish the same row.
- Partial indexes keep hot queries small: unpublished outbox rows (WHERE published_at IS NULL) and pending commands per entity (WHERE state = 'pending').
- Idempotency is enforced by unique keys: (tenant, idempotency_key) on orders and tasks, (tenant, id) on the inbox.
- Append-only tables (task_attempt, audit_event, orchestrator_inbox) have UPDATE revoked from dxps_app.

3.9 Useful queries

Run as dxps_ops for a cross-tenant view, or in a dxps_app transaction after SELECT set_config('app.tenant', 'host-mno', true). For one order, dxpsctl trace prints all of this as JSON without needing any password:

```
dxpsctl trace -tenant host-mno -order 1f2cd4a6-6333-5fbc-8fe1-0979d5966478

-- orders by tenant and state
SELECT tenant, state, count(*) FROM dxps.service_order GROUP BY 1, 2 ORDER BY 1, 2;

-- the task DAG of one order, in execution order
SELECT t.operation, t.state, t.attempt, t.is_compensation, t.ne_status, t.ne_code,
       array_agg(p.operation) FILTER (WHERE p.id IS NOT NULL) AS depends_on
FROM dxps.ne_task t
LEFT JOIN dxps.task_dependency d ON d.tenant = t.tenant AND d.task_id = t.id
LEFT JOIN dxps.ne_task p ON p.tenant = d.tenant AND p.id = d.depends_on_id
WHERE t.tenant = 'host-mno' AND t.order_id = '1f2cd4a6-6333-5fbc-8fe1-0979d5966478'
GROUP BY t.id ORDER BY t.created_at, t.seq;

-- NE latency over the last 15 minutes (partition pruning through the UUIDv7 bound)
SELECT ne.code, count(*), percentile_cont(0.99) WITHIN GROUP (ORDER BY a.latency_us) AS p99_us
FROM dxps.task_attempt a
JOIN dxps.ne_task t ON t.tenant = a.tenant AND t.id = a.task_id
JOIN dxps.network_element ne ON ne.tenant = t.ne_tenant AND ne.id = t.ne_id
WHERE a.id >= dxps.uuid_floor(now() - interval '15 minutes') GROUP BY 1 ORDER BY 3 DESC;

-- outbox backlog (should be 0 within seconds)
SELECT count(*) FROM dxps.outbox WHERE published_at IS NULL;

-- orders that have not finished after 10 minutes
SELECT tenant, id, state, requested_at FROM dxps.service_order
WHERE state IN ('acknowledged', 'inProgress') AND requested_at < now() - interval '10 minutes';
```

4. Apache Kafka 4 - asynchronous transport

4.1 What Kafka is used for

- Decoupling: the gateway, orchestrator, adapters and event hub never call each other; they exchange messages.
- Priority lanes: every command and task topic exists once per priority (p0 emergency, p1 interactive, p2 standard, p3 bulk), so bulk work cannot delay interactive orders.
- Ordering where it matters: records are keyed by tenant and entity, so everything for one subscriber on one NE stays in one partition, in order.
- Delayed retries without sleeping consumers (retry ladder topics), dead letters, and compacted state topics for dashboards.
- Local setup: one KRaft broker (no ZooKeeper) on 127.0.0.1:9092, controller on 9093, auto topic creation disabled; dxpsctl topics creates the 41 topics.

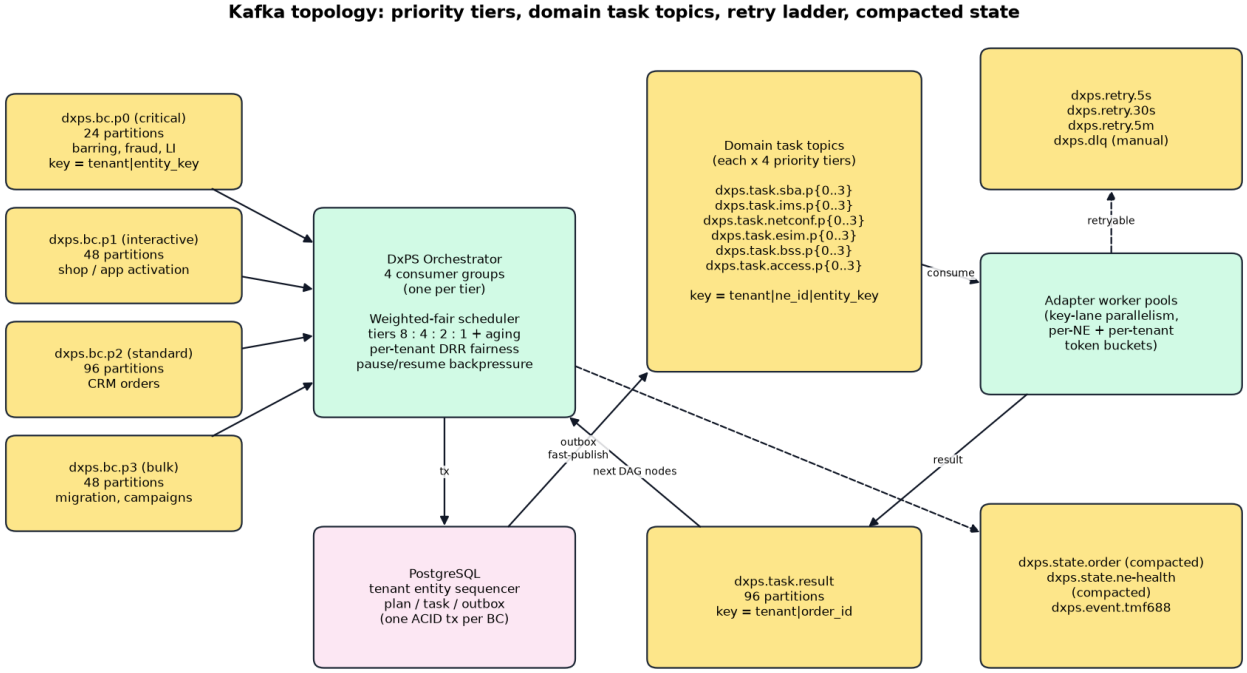


Figure: Kafka topology

4.2 Topic catalogue

Topics	Partitions	Retention / cleanup	Producer -> consumer group
dxps.bc.p0 .. p3 (4)	6	7 days	Gateway (via outbox) -> dxps-orchestrator-bc.pN
dxps.task.<domain>.p0 .. p3 (7 domains x 4 = 28)	3	broker default	Orchestrator (via outbox) -> dxps-adapter-<domain>-pN
dxps.task.result	6	broker default	Adapters (transactional) -> dxps-orchestrator-result.pN
dxps.retry.5s / 30s / 5m	3	broker default	Orchestrator -> dxps-retry-forwarder (republishes when due)
dxps.dlq	3	30 days	Any service -> dxpsctl redrive (dxps-dlq-redrive), operators
dxps.state.order	6	compacted	Orchestrator -> dashboards / downstream readers (latest state per order)
dxps.state.ne-health	1	compacted	Adapter -> dashboard
dxps.state.tenant	1	compacted	dxpsctl seed -> services (tenant snapshot)

dxps.event.tmf688	6	broker default	Orchestrator (via outbox) -> dxps-eventhub.pN

All topics: min.insync.replicas=1 (single broker), unclean leader election off, CreateTime timestamps, producer-side compression. For a production cluster, use the same layout with replication factor 3 and min.insync.replicas=2.

4.3 Keys and headers

Topic family	Record key	Why
dxps.bc.*	tenant|entity_key (host-mno|supi:imsi-416771008030236)	All commands for one subscriber land in one partition, in order
dxps.task.*	tenant|ne_id|entity_key	Calls for one subscriber on one NE are serial; different NEs run in parallel
dxps.task.result, dxps.event.tmf688, dxps.state.order, dxps.dlq	tenant|order_id	Results and events of one order are processed in order

Header	Example	Meaning
tenant	host-mno	Must equal the key prefix and the payload tenant, otherwise the record is rejected (DXPS-1004)
message_id	ff905945-8113-5e8d-9180-7df27917d9f4	Deterministic id; the orchestrator inbox drops duplicates
priority	p1	Lane of the record
schema_id	dxps.v1.BusinessCommand	Message contract and version
not_before / target_topic	(retry topics)	When and where the retry forwarder republishes the record
traceparent	W3C trace context	End-to-end tracing

4.4 Messages (real)

BusinessCommand on dxps.bc.p1 (written by the gateway in the order transaction):

```
key:     host-mno|supi:imsi-416771008030236
headers: tenant=host-mno  priority=p1  schema_id=dxps.v1.BusinessCommand  message_id=ff905945-8113-5e8d-9180-7df27917d9f4
{ "tenant": "host-mno", "message_id": "ff905945-8113-5e8d-9180-7df27917d9f4",
  "order_id": "1f2cd4a6-6333-5fbc-8fe1-0979d5966478", "order_item_id": "sub",
  "bc_id": "e602e652-1152-570a-87ac-533c5707d7b7",
  "entity_key": "supi:imsi-416771008030236", "extra_entity_keys": ["msisdn:962718030236"], "entity_seq": 0,
  "command_spec": "CreateSubscriber5G", "spec_version": "3.2.0", "action": "add",
  "priority": 1, "tx_mode": "ATOMIC", "channel": "DASHBOARD",
  "params": {"supi": "imsi-416771008030236", "msisdn": "962718030236", "plan": "LTE-20GB"} }
```

NeTask on dxps.task.sba.p1 (written by the orchestrator; the request is already rendered from the template):

```
key:     host-mno|01a113c4-d818-7b24-bb32-a35af5896bea|supi:imsi-416771008030236
headers: tenant=host-mno  priority=p1  schema_id=dxps.v1.NeTask  message_id=...
{ "tenant": "host-mno", "task_id": "01a11476-287d-7ed4-89e1-aaa76229497b",
  "order_id": "1f2cd4a6-6333-5fbc-8fe1-0979d5966478",
  "ne_tenant": "host-mno", "ne_id": "01a113c4-d818-7b24-bb32-a35af5896bea", "ne_code": "udr01",
  "domain": "sba", "operation": "udr.authSubscription.put", "entity_key": "supi:imsi-416771008030236",
  "attempt": 0, "priority": 1, "compensation": false,
  "idempotency_key": "host-mno:01a11476-287d-7ed4-89e1-aaa76229497b:0",
  "request": { "supi": "imsi-416771008030236", "authenticationMethod": "5G_AKA", ... } }
```

TaskResult on dxps.task.result (adapter -> orchestrator; values of the same call):

```
key: host-mno|1f2cd4a6-6333-5fbc-8fe1-0979d5966478
{ "tenant": "host-mno", "task_id": "01a11476-287d-7ed4-89e1-aaa76229497b",
  "order_id": "1f2cd4a6-6333-5fbc-8fe1-0979d5966478", "attempt": 0,
  "outcome": "SUCCEEDED", "ne_status": 201, "latency_us": 3819, "response": { ... } }
outcome: SUCCEEDED | RETRYABLE (5xx, timeout: retry ladder) | FAILED (4xx: compensation)
```

TMF688 event on dxps.event.tmf688, then delivered by the event hub (real webhook call):

```
POST https://localhost:9109/hooks/host-mno      (HTTP/2, 204 in 0.6 ms)
X-DxPS-Event-Id: 01a11476-2ce4-7d5f-bcdf-de0e52eeedbd
X-DxPS-Tenant:   host-mno
X-DxPS-Signature: <HMAC-SHA256 of the body with the subscription secret>
{ "eventId": "01a11476-2ce4-7d5f-bcdf-de0e52eeedbd", "eventType": "ServiceOrderStateChangeEvent",
  "eventTime": "2026-10-07T03:44:11.4928765Z",
```

```
"event": { "serviceOrder": { "id": "d635a915-6a95-5d96-b1a5-6e8418f4c286",
           "externalId": "DEMO-HOST-MNO-8030241", "state": "completed",
           "completionDate": "2026-10-07T03:44:11.4928765Z" } } }
```

4.5 Producers

Producer	Used for	Settings
fast	P0 / P1 records	acks=all, idempotent, lz4, no linger, 64 KB batches, 5 s delivery timeout, sticky key partitioner
bulk	P2 / P3 records	acks=all, idempotent, zstd, 10 ms linger, 1 MB batches, 120 s delivery timeout
transactional (adapter)	TaskResult + consumed offsets	transactional.id per domain, priority and instance; 30 s transaction timeout

4.6 Consumers

- Tiered consumer (orchestrator, event hub): one KIP-848 consumer group per priority tier (dxps-orchestrator-bc.p0 .. p3). Records go into the weighted-fair scheduler (P0 strict, P1:P2:P3 = 4:2:1, deficit round robin across tenants by SLA weight), then into the key-lane executor (same key serial, different keys parallel, up to 256). Only the highest contiguous completed offset per partition is committed, so a crash never skips a record.
- Exactly-once adapters: each adapter group (dxps-adapter-sba-p1, ...) consumes with read_committed, calls the NE, and produces the TaskResult and commits the consumed offset in one Kafka transaction.
- Retry ladder: a retryable failure goes to dxps.retry.5s, .30s or .5m with not_before and target_topic headers; dxps-retry-forwarder republishes it when due. After the maximum number of attempts the task fails with DXPS-4001.
- Dead letters: undecodable or tenant-mismatched records and failed tasks are written to dxps.dlq with an error header; dxpsctl redrive replays them to their source topic after a fix.
- Backpressure: the scheduler has a bounded capacity (4096); when full, polling stops instead of buffering without limit.

4.7 Outbox relay: database and Kafka in step

Services never produce to Kafka inside a database transaction. They insert outbox rows in the same transaction as the state change and, after commit, try an immediate publish (fast path). The relay in the orchestrator picks up anything older than 2 seconds that is still unpublished (crash, Kafka down) and publishes it with FOR UPDATE SKIP LOCKED. Delivery is therefore at-least-once; consumers make it effectively-once with the inbox, task attempt numbers and idempotency keys.

4.8 Volumes after the 2,000-order load run

Topic	Records	Topic	Records
dxps.task.result	16,229	dxps.task.bss.p1	1,417
dxps.state.ne-health	7,612	dxps.task.esim.p1	1,151
dxps.state.order	7,155	dxps.bc.p2	1,076
dxps.event.tmf688	7,155	dxps.task.ims.p1	827
dxps.task.sba.p1	4,447	dxps.dlq	373
dxps.bc.p1	2,550	dxps.retry.5s	336

All 36 DxPS consumer groups were Stable with lag 0 after the run.

5. The other components

5.1 Gateway (northbound API)

Endpoint (base /tmf-api/serviceOrdering/v5)	Scope	Use
POST /serviceOrder	dxps:order:write	Create an order (Idempotency-Key header supported)
GET /serviceOrder?state=&limit=&offset=	dxps:order:read	List the tenant's orders
GET /serviceOrder/{id}	dxps:order:read	Order with item states and errors
POST /cancelServiceOrder	dxps:order:write	Cancel a running order
POST /hub, DELETE /hub/{id}	dxps:hub:write	Register / remove a TMF688 webhook

- OAuth2 bearer JWT (HS256 only; iss, aud, exp, nbf and tenant required). The tenant comes from the token, never from the body.
- Per-tenant rate limit from tenant.quota_tps; suspended tenants are refused.
- Duplicate submissions with the same Idempotency-Key return the original order; a different body with the same key is a conflict.

5.2 Orchestrator

- Planner: resolves the spec (tenant override first, then GLOBAL), validates parameters, evaluates CEL conditions (for example policy only when the plan starts with 5G), selects NEs the tenant owns or is entitled to, and coalesces tasks on the same resource.
- Entity sequencer: commands on the same subscriber run one at a time in arrival order; different subscribers run in parallel.
- Saga: ATOMIC rolls back completed steps in reverse order; PARTIAL_ALLOWED keeps what succeeded; BEST_EFFORT never rolls back. A pivot task (the OCS account) marks the point after which the command must go forward.
- Order state is derived from the business commands: acknowledged -> inProgress -> completed / failed / partial / cancelled / rejected.

5.3 Catalog

A spec is YAML. This is the real CreateSubscriber5G (GLOBAL 3.2.0), abbreviated:

```
spec: CreateSubscriber5G
version: 3.2.0
txMode: ATOMIC
priorityDefault: P1
params:
  required: [supi, msisdn, plan]
  properties:
    supi:   {type: string, pattern: "^imsi-[0-9]{15}$"}
entityKeys: ["'supi:' + params.supi", "'msisdn:' + params.msisdn"]
tasks:
  - {id: auth,   ne: {nfType: UDR}, op: udr.authSubscription.put, template: sba/auth-subscription.json.tmpl,
     compensate: udr.authSubscription.delete}
  - {id: am,     dependsOn: [auth], op: udr.amData.put, coalesceKey: "'udr-am:' + params.supi", compensate:
udr.amData.delete}
  - {id: smf,    dependsOn: [auth], op: udr.smfSelection.put, compensate: udr.smfSelection.delete}
  - {id: policy, dependsOn: [auth], when: "params.plan.startsWith('5G')", op: udr.policyData.put}
  - {id: ocs,    dependsOn: [am, smf], ne: {nfType: OCS}, op: bss.account.create, pivot: true}
```

mvno-alpha overrides it (3.2.0-alpha.1) to use its own OCS and add a welcome bundle. Specs are loaded into command_spec at seed time; the version used is stored on every business command.

5.4 Adapter and network elements

NE	Owner	Domain / type	Simulator port	Entitled tenants (quota TPS)
nrf01	host-mno	sba / NRF (OAuth2 tokens)	9101	shared
udr01	host-mno	sba / UDR	9102	mvno-alpha (400), mvno-beta (150)

imspg01	host-mno	ims / IMS provisioning gateway	9103	mvno-alpha (200), mvno-beta (100)
pe01	host-mno	netconf / PE router (L3VPN)	9104	ent-acme (50)
olt01, usp01	host-mno	access / OLT (TR-385), USP controller (TR-369)	9104	host only
smdp01	host-mno	esim / SM-DP+ (ES2+)	9105	mvno-alpha (100), mvno-beta (50)
ocs01	host-mno	bss / OCS (TMF666)	9106	mvno-beta (150)
ocs-alpha	mvno-alpha	bss / OCS of the full MVNO	9106	mvno-alpha only
npdb01	host-mno	bss / number portability	9107	mvno-alpha (50), mvno-beta (50)
nef01	host-mno	exposure / NEF (CAMARA QoD)	9108	mvno-alpha (100), ent-acme (50)

- Before every call the adapter takes a token from the NE budget (max_tps, max_concurrency, reserved share for the host) and the tenant quota; a circuit breaker per NE opens after consecutive failures.
- All NE calls use HTTP/2 over mutual TLS with the DxPS CA. mvno-beta has no entitlement on nef01, which is why its QoDSession orders fail with DXPS-1005 before any call is made.

5.5 Network simulators

netsim implements every southbound interface with payload validation and per-tenant state (a subscriber created by one tenant cannot be changed by another). The admin port 9199 sets latency and error rates. Test inputs trigger faults: a parameter ending in 997 gives a transient 503, 998 a permanent 4xx rejection, 999 an NE that is down; a USP device marked OFFLINE returns 7002; a QoD bandwidth above 10,000 returns 409.

5.6 Event hub

- Consumes dxps.event.tmf688 and posts each event to the tenant's registered callbacks.
- Callbacks must be https and allow-listed; redirects are not followed; delivery is retried with backoff.
- Each subscription has its own secret, stored encrypted (AES-256-GCM, bound to the tenant) and used for the X-DxPS-Signature HMAC.

5.7 Mosaic dashboard

Reads statistics from every service's loopback stats port, Kafka (topic end offsets, group lag) and PostgreSQL (as dxps_ops: orders by state, tasks by domain, NE latency percentiles, outbox backlog), and shows them as live tiles. It also contains a load generator and security probes.

5.8 DxPS console

The console is the signed-in tool for operating and configuring DxPS across all tenants. It has two parts: the console API (Go, https://localhost:8446, loopback only) and the web UI (Next.js, https://localhost:8092). The UI server forwards every call to the API; the session token travels in an HttpOnly, SameSite=Strict cookie that page scripts cannot read, and every change must carry a CSRF header. Users have one or more of four roles: viewer, operator (orders, faults, DLQ redrive, load runs), configurator (tenants, NE inventory, catalog drafts, Template Lab) and admin (users).

Data	Where	How the console uses it
Orders, tasks, attempts, NE inventory, audit	PostgreSQL, as dxps_ops	Reads across all tenants: lists, order detail with the task graph and the NE request and response of every attempt, reports
tenant, network_element, tenant_ne_access	PostgreSQL, as dxps_ops	Updates tenant SLA tier, weight, quota and status, NE limits, and entitlements; each change writes an audit_event row with actor console:<user> and the before and after values
Topics and consumer groups	Kafka	End offsets and group lag; dead letters are read from dxps.dlq and redriven to their original topic
Service statistics	Stats ports 9201-9203, Mosaic 8088, netsim admin 9199	Health, live rates, simulator latency and fault settings
New orders	Gateway 8443	Submitted and cancelled through the TMF641 API with a 2-minute tenant token limited to order read and write
Console users, drafts, console audit	~/dxps-runtime/console/	users.json (PBKDF2-SHA256 hashes, mode 0600), drafts/ and audit.jsonl

Catalog changes use drafts because the services run with the catalog compiled in. A draft holds changed or new spec files. Validate loads the whole catalog with the change and runs the example of every spec through the Template Lab. The Lab runs the real planner and adapter renderers without calling any NE and shows the task graph, the CEL decisions and the exact NE requests. Export produces the files to copy into internal/catalog/data before a rebuild.

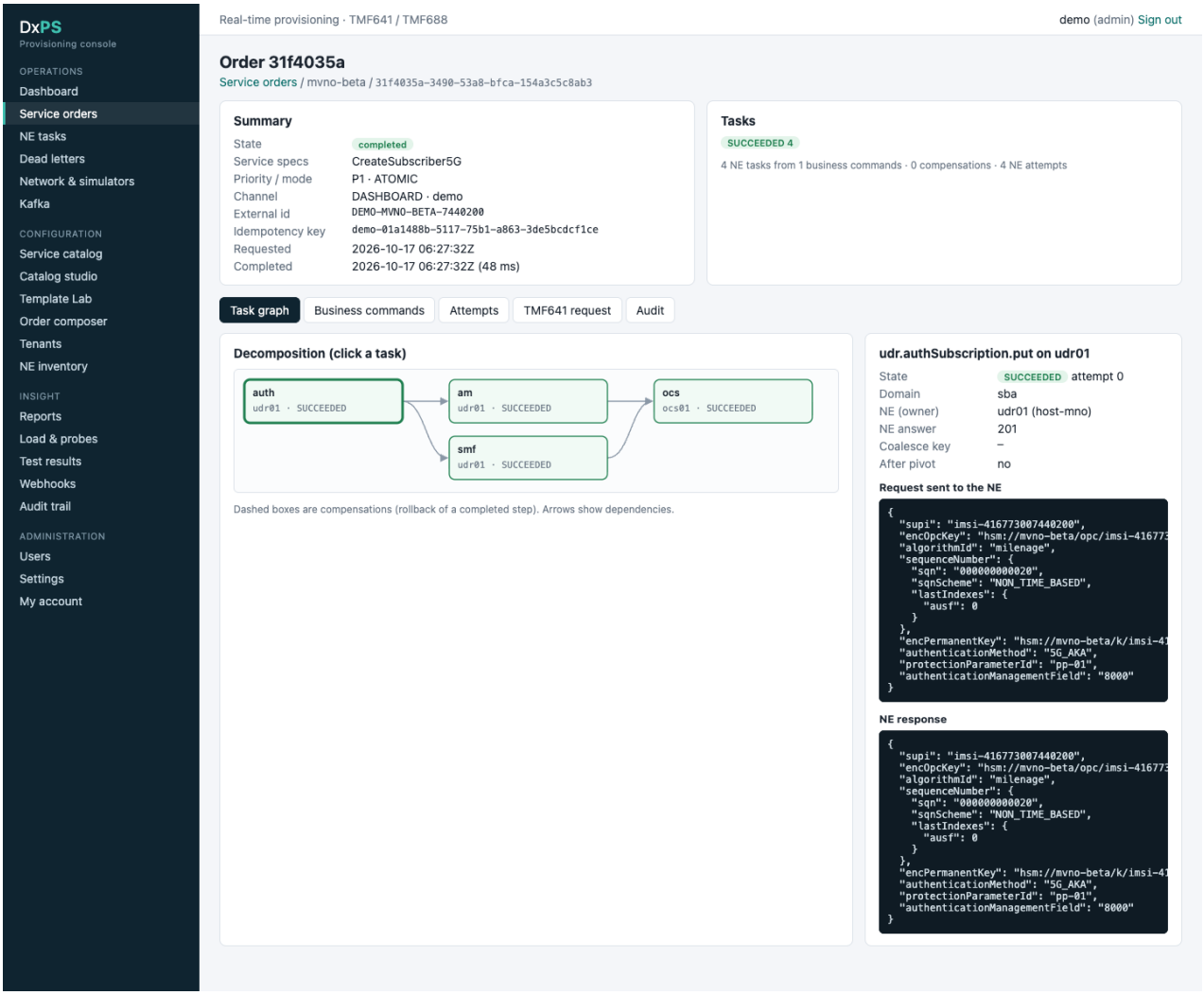


Figure: Console: order detail with the task graph and NE exchange

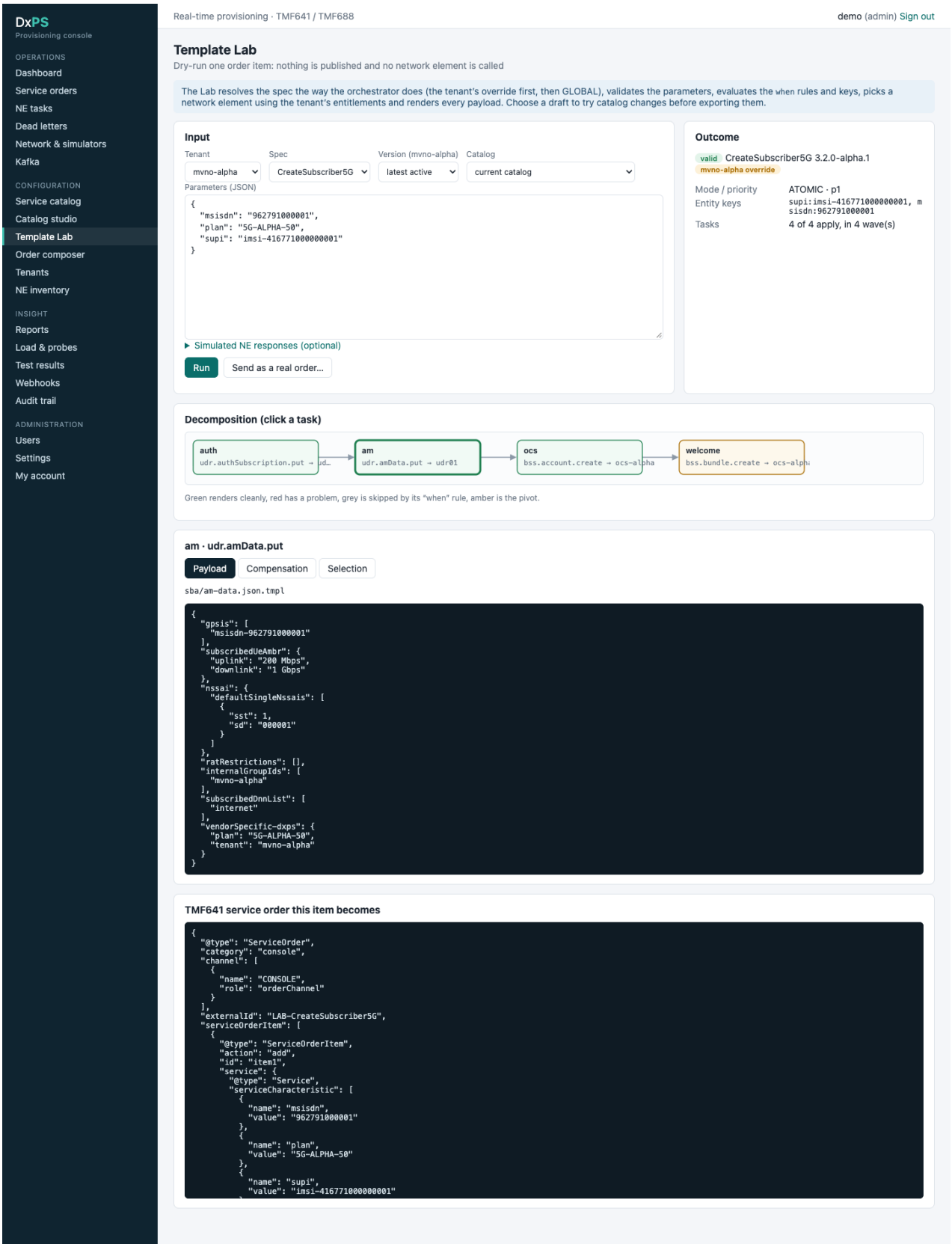


Figure: Console: Template Lab dry run

5.9 PKI, secrets and dxpsctl

- dxpsctl secrets generates database passwords, the JWT key and the secret-box key into secrets.env (mode 0600) in the runtime directory.
- dxpsctl certs creates the development CA and ECDSA P-256 certificates for PostgreSQL, the gateway, the services and every simulator.
- dxpsctl migrate / seed / topics prepare PostgreSQL and Kafka; dxpsctl api and trace give read access without handling passwords.

6. What happens when something fails

Situation	What protects the order	Component
Client sends the same order twice	Idempotency-Key -> same order id (UUIDv5); unique (tenant, idempotency_key)	Gateway, PostgreSQL
Service crashes after commit, before publishing	Outbox row is still unpublished; the relay publishes it	PostgreSQL outbox, relay
Kafka unavailable	Orders are still accepted and stored; the outbox drains when Kafka is back	Outbox
Message delivered twice	Inbox (message_id), attempt numbers and NE idempotency keys make the repeat a no-op	Orchestrator, adapter
NE timeout or 5xx	Retry ladder 5 s / 30 s / 5 min, then DXPS-4001 and compensation	Orchestrator, retry forwarder
NE rejects the request (4xx)	ATOMIC: reverse-order compensation; order failed with the NE error	Saga
Adapter crashes mid-call	Offset was not committed (transaction aborted): the task is redelivered with the same idempotency key	Kafka EOS
Two orders change the same subscriber	Entity sequencer runs them one after the other	Orchestrator, PostgreSQL
A tenant floods the system	Gateway rate limit, tier weights, per-tenant DRR and NE quotas	Gateway, scheduler, registry
Forged tenant in a message	Header, key prefix and payload tenant must match; RLS blocks cross-tenant rows	All consumers, PostgreSQL